

VoxIntel Phase 2

Detailed Documentation: Step 11 (Final Dataset Quality Filtering), Step 12 (Revalidation + Split Recreation) & Step 13 (QLoRA Fine-tuning Setup)

This document explains, in detail, exactly what was changed in the project during steps 11, 12, and 13 of the fine-tuning pipeline, why each change was made, what the results were, and how to open and run the project in Visual Studio Code. It is written so it can be used directly when explaining the project to others (e.g. a reviewer, a professor, or teammates).

1. Pipeline Overview

The VoxIntel Phase 2 project fine-tunes Qwen3-4B on a Tulu-English dictionary and conversation dataset. The full pipeline has 16 steps. This document covers steps 11, 12, and 13, which sit between dataset preparation and running the actual QLoRA fine-tuning job.

#	Step	Status
1	Dictionary cleaning	Done (prior)
2	Dictionary validation	Done (prior)
3	Instruction dataset generation	Done (prior)
4	Conversation dataset preparation	Done (prior)
5	Dataset merging	Done (prior)
6	Dataset improvement / cleaning	Done (prior)
7	Training dataset validation	Done (prior)
8	Train/validation/test split	Done (prior)
9	Python environment setup	Done (prior)
10	Qwen3-4B tokenizer test	Done (prior)
11	Final dataset quality filtering	COMPLETED
12	Revalidate + recreate splits	COMPLETED
13	QLoRA fine-tune Qwen3-4B	SCRIPT READY — needs to be run on a GPU machine
14	Evaluate the fine-tuned model	Pending
15	Integrate model with VoxIntel	Pending
16	Final testing + documentation	Pending

2. Step 11 — Final Dataset Quality Filtering

2.1 Why this step was needed

Step 7 (training dataset validation) had already scanned the merged/cleaned dataset (**final_training_dataset_cleaned.jsonl**, 60,778 records) and flagged problems, but it only **reported** them — nothing was actually fixed or removed. Re-running that validation script confirmed **4,002 records** were still flagged as suspicious:

Issue detected	Records affected
----------------	------------------

Contains English/Latin characters mixed into Tulu/Kannada responses (scan / OCR noise, e.g. "half-grown-", "AR,", "Thriving,.")	2,573
Contains repeated punctuation (e.g. "...", ".,", ",,,")	1,834

If left in the dataset, this noise would have been learned by the fine-tuned model — it would occasionally output stray English fragments or malformed punctuation inside Tulu answers.

2.2 What was built

A new script, **scripts/filter_final_dataset.py**, was added to the project. It reads the step-6/7 output and produces a cleaned, filtered dataset ready for re-splitting. It performs the following operations, in this exact order:

- 1 Remove leftover dictionary abbreviations (adj., adv., v., n., s., pron., prep., conj., interj., aux., cf.) that slipped past earlier cleaning.
- 2 Remove stray Latin/English tokens embedded inside Tulu/Kannada responses — any whitespace-separated token containing an English letter is dropped, since in this Tulu dictionary, English fragments in the response field are scan noise, not intended content.
- 3 Collapse repeated / mixed punctuation runs (e.g. "...", ".,", ",,,", "...") down to a single punctuation mark.
- 4 Normalize punctuation spacing and collapse extra whitespace.
- 5 Run the punctuation-collapse step a second time after spacing normalization — this was necessary because removing a space before punctuation could re-glue two previously separate marks back together (e.g. ". ." → "."), re-introducing the exact issue step 2.2's third bullet had just fixed. This was caught and fixed during testing (see section 2.4).
- 6 Strip stray leading/trailing punctuation left at the edges of a response.
- 7 If, after all cleaning, a response is empty or shorter than 2 characters (i.e. it was *entirely* noise, such as a response that was only "y" or "_"), the record is removed — it cannot be salvaged automatically.
- 8 Check for new exact duplicate (instruction, response) pairs created as a side effect of cleaning, and remove them.
- 9 Every removed record is logged (not silently deleted) to **output/removed_records_review.jsonl** with the reason it was removed, so it can be manually spot-checked later.

2.3 Files added / changed in step 11

File	Type	Purpose
scripts/filter_final_dataset.py	New script	Performs the step 11 cleaning/filtering logic described above.
output/final_training_dataset_filtered.jsonl	New output	The cleaned, filtered dataset (60,074 records). This is the file used going forward.
output/removed_records_review.jsonl	New output	The 704 records that were dropped, with reasons, for manual review.

output/final_filtering_report.txt	New output	Plain-text summary report of the filtering run (counts, before/after quality checks).
STEP11_NOTES.md	New doc	Markdown notes summarizing what step 11 does and its results.

2.4 A bug that was found and fixed during testing

While testing the script against the real dataset, the first version left 50-53 records with repeated punctuation still present, even though the collapsing logic looked correct in isolation. Root cause:

normalize_punctuation_spacing() removes the space before a punctuation mark (e.g. turns "word ." into "word."). When two separate punctuation marks were originally separated only by whitespace (e.g. "text. ."), this spacing step glued them back together into ".." — *after* the punctuation-collapsing step had already run. The fix was to run punctuation-collapsing a second time, after spacing normalization, as a final pass. After the fix, re-validation confirmed 0 records with repeated punctuation and 0 records with English/Latin characters remaining.

2.5 Results

Metric	Value
Records loaded	60,778
Records kept (filtered dataset)	60,074
Records removed — empty/too short after cleaning	704
Records removed — new duplicates created by cleaning	0
Records auto-fixed (text changed but kept)	4,446
Remaining English/Latin characters in responses (after fix)	0
Remaining repeated punctuation (after fix)	0

Example of an automatic fix applied by step 11:

```
Instruction: "Tulu word for Wrist."
Before: "■■■■■■■■■■ ■■■■ AR"
After: "■■■■■■■■■■ ■■■■"
```

3. Step 12 — Revalidate + Recreate Splits

3.1 What this step does

Step 12 has two parts: (a) confirm the step 11 output actually passes validation, and (b) regenerate the train/validation/test splits from the new filtered dataset, since the old splits were built from the unfiltered, noisier dataset.

3.2 Part A — Revalidation

The existing **scripts/validate_training_dataset.py** (originally used for step 7) was reused, with its input path updated from `final_training_dataset_cleaned.jsonl` to `final_training_dataset_filtered.jsonl`. Its output file names were also changed (to `training_dataset_revalidation_report.txt` and `suspicious_training_records_after_filtering.jsonl`) so the original step-7 report is preserved for comparison rather than being overwritten.

Check	Step 7 result (before filtering)	Step 12 result (after filtering)
Valid JSON records	60,778	60,074
Missing / empty fields	0	0
Duplicate records	0	0
Very short / very long records	0	0
Suspicious records (quality warnings)	4,002	0
Overall status	PASSED (with warnings)	PASSED (no warnings)

3.3 Part B — Recreating the splits

The existing **scripts/split_training_dataset.py** (originally used for step 8) was reused, with only its input path changed from `final_training_dataset_cleaned.jsonl` to `final_training_dataset_filtered.jsonl`. No other logic was touched, so the split methodology is unchanged and still performs:

- A 90% / 5% / 5% train / validation / test split (unchanged ratios).
- Grouping of dictionary-style examples by their headword (e.g. all examples for the word "Ability" are kept together) so the same word can never leak across splits.
- A safety check for exact (instruction, response) duplication leakage between splits.
- A safety check for dictionary-word leakage between splits.
- A fixed random seed (42) so the split is reproducible.

3.4 Files changed / regenerated in step 12

File	Type	Purpose
<code>scripts/validate_training_dataset.py</code>	Modified	Input path repointed to the filtered dataset; output file names changed so step 7's original report is kept.
<code>scripts/split_training_dataset.py</code>	Modified	Input path repointed to the filtered dataset. Split logic itself unchanged.
<code>output/training_dataset_revalidation_report.txt</code>	New output	Confirms 0 suspicious records remain after filtering.
<code>output/suspicious_training_records_after_filtering.jsonl</code>	New output	Empty — no suspicious records found.

data/training/train.jsonl	Regenerated	New training split, built from the filtered dataset (54,054 records).
data/training/validation.jsonl	Regenerated	New validation split (2,999 records).
data/training/test.jsonl	Regenerated	New test split (3,021 records).
data/training/split_statistics.txt	Regenerated	Updated split statistics and leakage checks.

3.5 Results

Split	Records	% of total	Unique dictionary words
Train	54,054	89.98%	10,808
Validation	2,999	4.99%	600
Test	3,021	5.03%	601
Total	60,074	100.00%	12,009

Leakage checks: 0 exact-duplicate overlaps between any pair of splits, and 0 dictionary-word overlaps between any pair of splits. Both step-12 goals — revalidating the cleaned data and recreating leak-free splits — are confirmed complete.

4. Step 13 — QLoRA Fine-tune Qwen3-4B

4.1 Important limitation — please read first

Unlike steps 11 and 12, step 13 could **not be executed and verified end-to-end** in the working environment used to build this project. Fine-tuning a 4-billion parameter model requires a CUDA-capable GPU, and downloading the base model requires an internet connection to Hugging Face — neither was available. Instead, a complete, syntax-checked training script was written and is ready to run on any machine with a GPU (your own PC, Google Colab, Kaggle, RunPod, Lambda Cloud, etc.). This is the single most important thing to be able to explain about this step: **the code is done, but the actual training run still needs to happen on GPU hardware.**

4.2 What QLoRA is, briefly

QLoRA ("Quantized Low-Rank Adaptation") is a memory-efficient fine-tuning method. Instead of updating all ~4 billion parameters of Qwen3-4B (which would need well over 48 GB of VRAM), it: (1) loads the base model compressed to 4-bit precision so it fits in a fraction of the memory, and (2) freezes those weights entirely, training only a small set of extra "adapter" matrices (LoRA) that are inserted into each attention and MLP layer. Typically under 1% of the model's total parameters are actually trained. This is what makes fine-tuning a 4B model realistic on a single consumer GPU.

4.3 What the script does

- 1 Loads Qwen/Qwen3-4B in 4-bit precision (NF4 quantization, double quantization, bfloat16 compute type) using bitsandbytes.
- 2 Attaches LoRA adapters (rank 16, alpha 32, dropout 0.05) to all seven attention/MLP projection layers (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj).
- 3 Loads data/training/train.jsonl (54,054 records) and validation.jsonl (2,999 records) — the exact files produced in step 12.
- 4 Formats every {instruction, response} pair using Qwen's own chat template, so the model sees data in the same format it will be used in at inference time.
- 5 Masks the loss on the prompt/instruction tokens, so the model is trained only to predict the Tulu response — not to reproduce the question back.
- 6 Trains with the Hugging Face Trainer for 3 epochs by default, evaluating on the validation set periodically and automatically keeping the checkpoint with the lowest validation loss.
- 7 Saves the final LoRA adapter to models/qwen3-4b-tulu-lora/final_adapter/ plus a training_summary.json recording the settings and final loss used for that run.

4.4 Key hyperparameters

Setting	Default	Notes
Epochs	3	Reasonable starting point for ~60k examples

Per-device batch size	4	Lower this first if you hit out-of-memory errors
Gradient accumulation steps	4	Effective batch size = $4 \times 4 = 16$
Learning rate	2e-4	Typical for LoRA (higher than full fine-tuning)
LoRA rank (r)	16	Higher = more capacity, more VRAM, slower
Max sequence length	512	Dictionary entries are short; safe margin

4.5 Tuning for your GPU's VRAM

GPU VRAM	Example cards	Suggested change
8 GB or less	RTX 3060 Ti, RTX 4060	batch size = 1, gradient accumulation = 16
12-16 GB	RTX 3060 12GB, RTX 4070 Ti, RTX 4080	defaults should work as-is
24 GB+	RTX 3090 / 4090, A5000	batch size 8-16, lower gradient accumulation

Rule of thumb: keep (batch size) x (gradient accumulation steps) roughly constant when adjusting for your hardware, to keep training dynamics similar to the defaults.

4.6 Files added / changed in step 13

File	Type	Purpose
scripts/finetune_qlora.py	New script	The main QLoRA training script described above.
scripts/quick_test_adapter.py	New script	Loads the trained adapter and runs a few Tulu prompts through it as a fast sanity check, before the full step 14 evaluation.
scripts/config.py	Modified	Added TRAINING_DATA and MODELS_DIR path constants.
requirements.txt	Modified	Added torch, peft, bitsandbytes, accelerate, datasets; raised transformers to $\geq 4.51.0$ (first version with Qwen3 support).
STEP13_NOTES.md	New doc	Markdown notes summarizing step 13 (mirrors this section).

4.7 How to actually run step 13

- 1 Get access to a machine with an NVIDIA GPU — your own, or a cloud notebook (Google Colab with a T4/A100 runtime is the easiest free/cheap option to start with).
- 2 Copy the whole VoxIntel_Phase2 project folder to that machine.
- 3 Install dependencies: `pip install -r requirements.txt` (this now includes torch, peft, bitsandbytes, accelerate, and datasets).
- 4 Make sure you're logged in to Hugging Face if Qwen3-4B requires it (huggingface-cli login), since the script downloads the base model on first run.

- 5 Run: `python scripts/finetune_qlora.py` — this will take anywhere from under an hour to several hours depending on your GPU.
- 6 Once training finishes, run: `python scripts/quick_test_adapter.py` to eyeball a few sample translations before moving to the full step 14 evaluation.

5.1 One-time setup

- 1 Install VS Code (code.visualstudio.com) if not already installed.
- 2 Install the official **Python** extension from Microsoft inside VS Code (Extensions panel → search "Python" → Install).
- 3 Extract/copy the project folder VoxIntel_Phase2 somewhere on your machine.
- 4 In VS Code: **File** → **Open Folder...** → select the VoxIntel_Phase2 folder.

5.2 Create a virtual environment (recommended)

Open a terminal inside VS Code (**Terminal** → **New Terminal**), then run:

```
python -m venv venv
# Windows:
venv\Scripts\activate
# macOS / Linux:
source venv/bin/activate
```

After activating, press **Ctrl+Shift+P** (Cmd+Shift+P on Mac) → type "Python: Select Interpreter" → choose the interpreter inside the new venv folder, so VS Code runs scripts with the right environment.

5.3 Install dependencies

The project's requirements.txt was originally empty and was filled in during step 11. It now also includes the packages needed for step 13 (added in this session): torch, peft, bitsandbytes, accelerate, and datasets, with transformers pinned to >=4.51.0 for Qwen3 support. Install everything with:

```
pip install -r requirements.txt
```

Note: torch + bitsandbytes require a CUDA-capable GPU to actually be useful for training (step 13). They will install fine on a CPU-only machine, but finetune_qlora.py will refuse to run without a detected GPU, since CPU training of a 4B parameter model is not practical.

5.4 Running the scripts

All scripts are run from the project's **root folder** (the folder containing "scripts", "data", "output"), not from inside the scripts folder, since paths in config.py are relative to the project root. In the VS Code terminal:

```
cd VoxIntel_Phase2
python scripts/filter_final_dataset.py # Step 11
python scripts/validate_training_dataset.py # Step 12 (part A - revalidate)
python scripts/split_training_dataset.py # Step 12 (part B - recreate splits)
python scripts/finetune_qlora.py # Step 13 (needs a CUDA GPU)
python scripts/quick_test_adapter.py # Step 13 sanity check, after training
```

Each script prints a summary to the terminal and also writes a report file into the output/ or data/training/ folder, so results can be checked either in the terminal or by opening the generated file directly in VS Code's editor.

5.5 Quick way to inspect the .jsonl output files

JSONL files can be opened directly in VS Code (they're just plain text, one JSON object per line). For nicer viewing, the "JSONL" extension by "mikestead" adds syntax highlighting. To check record counts from the terminal:

```
wc -l output/final_training_dataset_filtered.jsonl
wc -l data/training/train.jsonl data/training/validation.jsonl data/training/test.jsonl
```

5.6 Recommended run order for the whole pipeline (for reference)

Step	Script
1-2	clean_dictionary.py, fix_dictionary.py, standardize_dictionary.py, validate_dictionary.py
3	generate_instruction_dataset.py
4	prepare_conversation_dataset.py
5	merge_datasets.py
6	improve_training_dataset.py
7	validate_training_dataset.py (original run, against cleaned dataset)
8	split_training_dataset.py (original run, against cleaned dataset)
9-10	environment setup, test_tokenizers.py
11	filter_final_dataset.py (completed)
12	validate_training_dataset.py + split_training_dataset.py (re-run, against filtered dataset) (completed)
13	finetune_qlora.py (script ready - run on a GPU machine), then quick_test_adapter.py
14+	Evaluation, integration, final docs (not yet built)

Document generated to accompany the VoxIntel_Phase2 project zip after completion of steps 11 and 12.